Table 1: Revision History

Date	**Developer(s)**	**Change**
April 5, 2026	Savinay	Initial User Manual release with interface startup, TUI/GUI settings, MCP setup, and Discord integration guidance
April 6, 2026	Savinay	Added `uv run proxi setup` installation flow and documented the GUI voice controls

User Manual
Project Proxi

Team #23, Project Proxi
Savinay Chhabra
Amanbeer Singh Minhas
Gourob Podder
Ajay Singh Grewal

1 Overview

This manual describes the recommended workflow for Proxi:

1. Start exactly one gateway process
2. Choose either the TUI or the GUI as your interface
3. If you use the GUI, configure settings (API keys, Integrations, and cron jobs)

2 Installation

From the repository root, install dependencies and initialize the local Proxi setup with:

```
uv sync
uv run proxi setup
```

This one-time setup flow:

- Installs Python dependencies with `uv`
- Installs the Node dependencies needed by the TUI, GUI, and Discord relay
- Initializes or verifies the local API key database in `config/api_keys.db`

If you ever need to repair a local environment, rerun `uv run proxi setup`. It is safe to run more than once.

3 Starting the Interface

This section describes the required startup order and your three interface options.

3.1 Step 1 (Always): Start the Gateway

Run:

```
uv run proxi gateway start
```

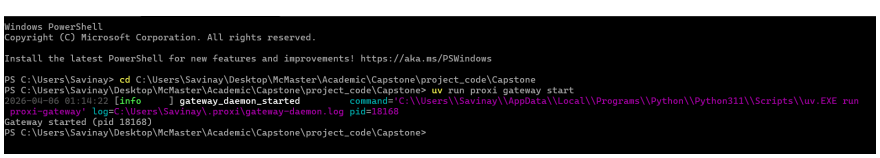

Figure 1: Gateway startup command in terminal.

3.2 Step 2: Choose an Interface

3.2.1 Option A: TUI

Run:

```
uv run proxi
```

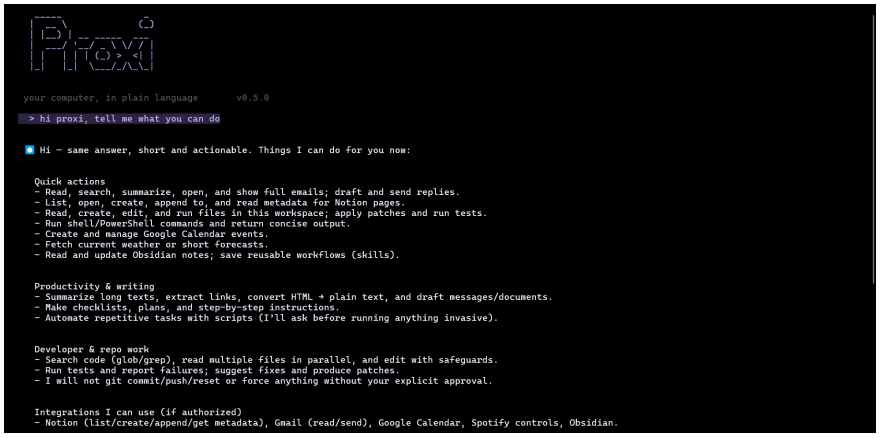

Figure 2: Proxi TUI interface after launch.

3.2.2 Option B: GUI (React Frontend)

Run:

```
uv run proxi frontend
```

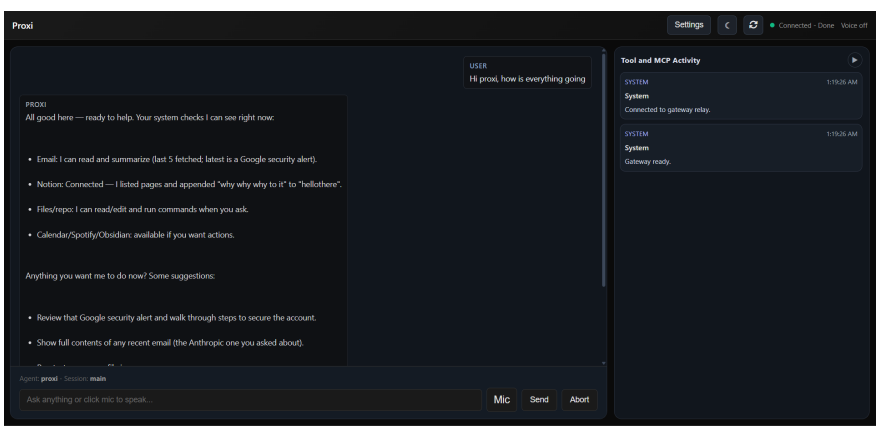

Figure 3: Proxi GUI home screen.

3.2.3 Option C: Discord

Run:

```
uv run proxi discord
```

Then send commands via connected Discord bot.

4 TUI Interface and Settings

This section explains only TUI usage and TUI-specific settings/actions.

4.1 Launch and Connect

Start the TUI:

```
uv run proxi
```

If not already running, start gateway first:

```
uv run proxi gateway start
```

4.2 Opening the Command Palette

In the TUI input box, type:

```
/
```

- Type to filter commands
- Use up/down arrows to select
- Press Enter to execute
- Press Esc to close the palette

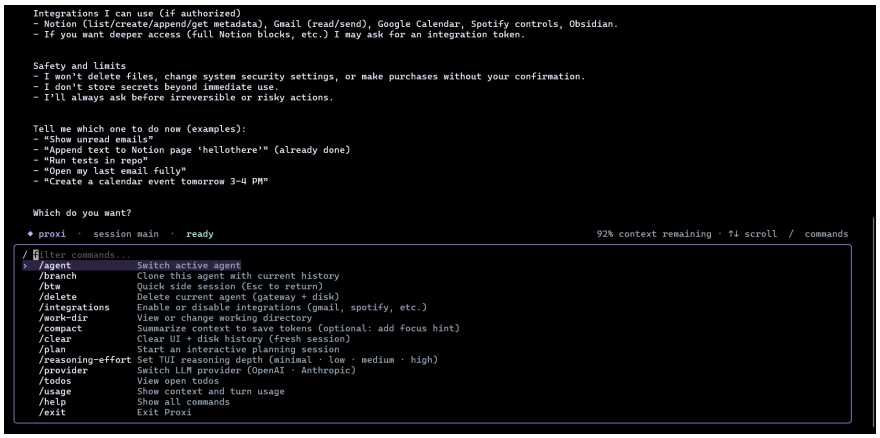

Figure 4: TUI slash command interface and command discovery flow.

4.3 Complete Slash Command Reference

Table 2: TUI Slash Commands

Command	What it does
`/agent`	Switch active agent or create a new one
`/switch-agent`	Alias for `/agent`
`/delete`	Delete current agent (gateway configuration and agent workspace)
`/mcps`	Open MCP management to enable/disable integrations
`/work-dir`	View or change the working directory used by coding/file tools
`/clear`	Clear chat UI and current session history for a fresh session
`/plan`	Open/view current session plan
`/todos`	Open/view current session todos
`/usage`	Show context-window and turn usage statistics
`/help`	Print command reference in chat
`/exit`	Exit the TUI

4.4 When to Use Which Command

- Use `/agent` when switching between workflows (for example, planning vs coding)
- Use `/mcps` after adding keys to turn integrations on/off
- Use `/work-dir` before coding tasks to constrain file/tool scope
- Use `/clear` when you want a clean context for a new task

- Use `/plan` and `/todos` to inspect execution state
- Use `/usage` to monitor context usage and avoid overflow

4.5 TUI Settings Summary

In TUI mode, operational settings are primarily controlled by slash commands:

- Agent setting: `/agent`
- MCP enable/disable: `/mcps`
- Working directory scope: `/work-dir`
- Session reset: `/clear`

5 GUI Interface and Settings

Open **Settings** from the GUI top bar. The settings modal is organized into these sections:

1. General
2. Agent
3. Profile
4. API Keys
5. Integrations
6. Cron Jobs
7. Webhooks
8. Voice

5.1 General

Use this section for quick session and model controls.

5.1.1 What You Can Configure

- **Clear History**: clears current session history
- **Reasoning Effort**: set reasoning depth (minimal, low, medium, high)
- **Compact Focus Hint**: optional hint used when compacting context
- **Provider and Model**: select active LLM provider/model for new turns

5.1.2 Suggested Usage

- Use **Clear History** before a new task domain
- Use **Reasoning Effort** to balance speed vs depth
- Use **Compact** when context gets large
- Use **Switch LLM Provider** after selecting provider/model

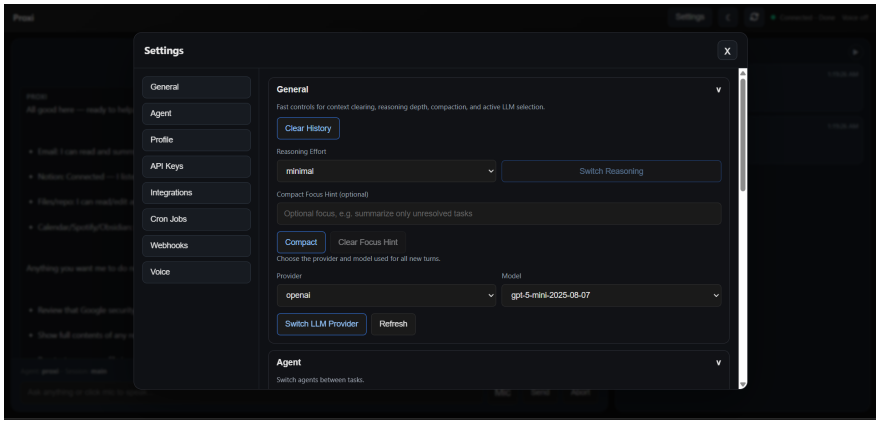

Figure 5: GUI Settings: General section.

5.2 Agent

Use this section to control which agent receives your prompts.

5.2.1 Fields and Actions

- **Current Agent**: read-only indicator of active agent ID
- **Agent Selector**: choose another configured agent
- **Switch Agent**: apply selected agent

5.2.2 Recommended Workflow

1. Select the agent that matches your use case (for example planning vs coding)
2. Switch agent before starting a new workflow
3. Verify in chat that responses come from the expected persona/behavior

5.3 Profile

Use this section to personalize assistant behavior.

5.3.1 Important Fields

- Name and Occupation: style/context personalization
- Location and Timezone: local-time suggestions and schedule accuracy
- Email and Email Signature: outbound drafting quality
- Demographics: optional context used by selected prompts

5.3.2 Best Practices

- Keep timezone accurate if you use calendar/cron automations
- Keep email signature short and reusable
- Update profile when switching environments (school/work/personal)

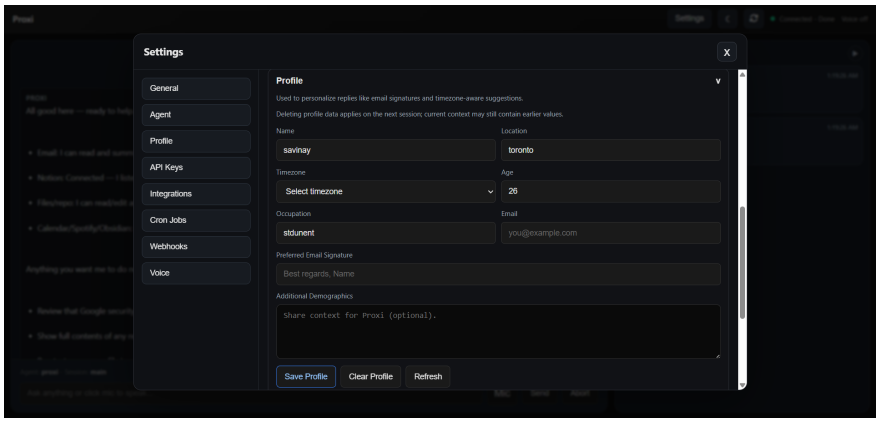

Figure 6: GUI Settings: Profile section.

5.4 API Keys

Use this section to add integration credentials and model-provider keys.

5.4.1 MCP Key Mapping Table

Use the following key names exactly as written (do not include = when entering key names in the GUI).

Table 3: MCP and Provider Key Mapping

MCP / Provider	Key Name(s)
LLM Provider (OpenAI)	`OPENAI_API_KEY`
Google Calendar	`GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET`
Gmail	`GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET`
Notion	`NOTION_API_KEY`, `NOTION_PARENT_PAGE_ID`
Spotify	`SPOTIFY_CLIENT_ID`, `SPOTIFY_CLIENT_SECRET`
Discord relay/config (if used)	`DISCORD_BOT_TOKEN`, `PROXI_DISCORD_COMMAND_PREFIX`
Obsidian	No API key required (vault/path configuration)
Weather	Usually no API key required in this setup

5.4.2 Notes

- `OPENAI_API_KEY` is the primary LLM key in this setup
- Keep secrets private and do not commit real values to version control
- After adding keys, refresh and verify they are listed

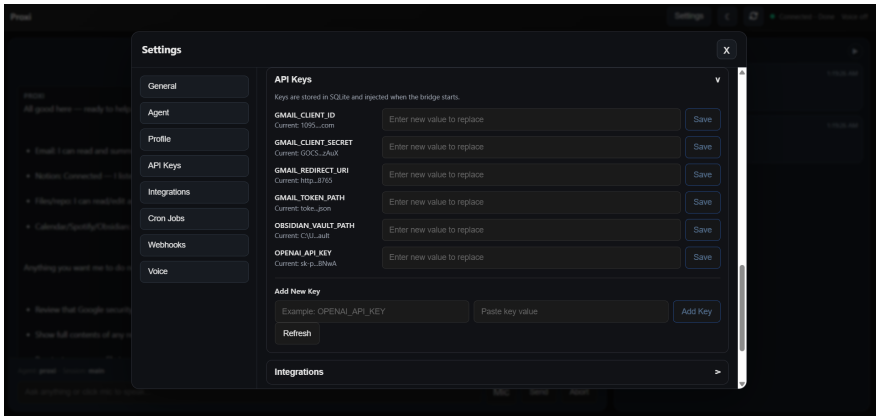

Figure 7: GUI Settings: API Keys section.

5.5 Integrations

Use this section to enable or disable integrations after key setup.

5.5.1 General Enablement Steps

1. Open **Settings > Integrations**

2. Toggle required integrations to enabled
3. Save/refresh
4. Run one test prompt per integration

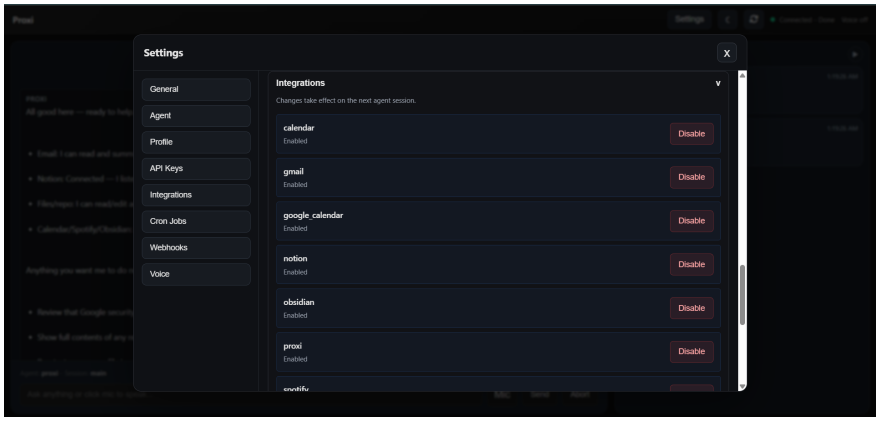

Figure 8: GUI Settings: Integrations section.

5.6 Cron Jobs

Use this section to schedule recurring prompts.

5.6.1 Core Fields

- Source ID
- Target Agent
- Target Session (optional)
- Priority
- Prompt
- Schedule

5.6.2 Easy Schedule (Recommended)

If you do not want to write cron patterns manually:

- Set **Every** (number)
- Set **Unit** (Minute, Day, Week, Monthly)
- Let the UI generate cron expression automatically

Examples:

- Every 30 Minute
- Every 1 Day
- Every 1 Week

5.6.3 Manual Cron (Advanced)

Example expression:

`*/30 * * * *`

Syntax meaning (5-field cron format):

- Field 1 = minute
- Field 2 = hour
- Field 3 = day of month
- Field 4 = month
- Field 5 = day of week

For `*/30 * * * *` specifically:

- `*/30` in the minute field means "every 30 minutes" (minute 0 and 30)
- Each `*` means "any value" for that field
- So the schedule runs every hour, every day, every month

Additional examples:

- `0 9 * * *`: run every day at 9:00
- `0 */4 * * *`: run every 4 hours on the hour
- `30 17 * * 1-5`: run at 17:30 on weekdays

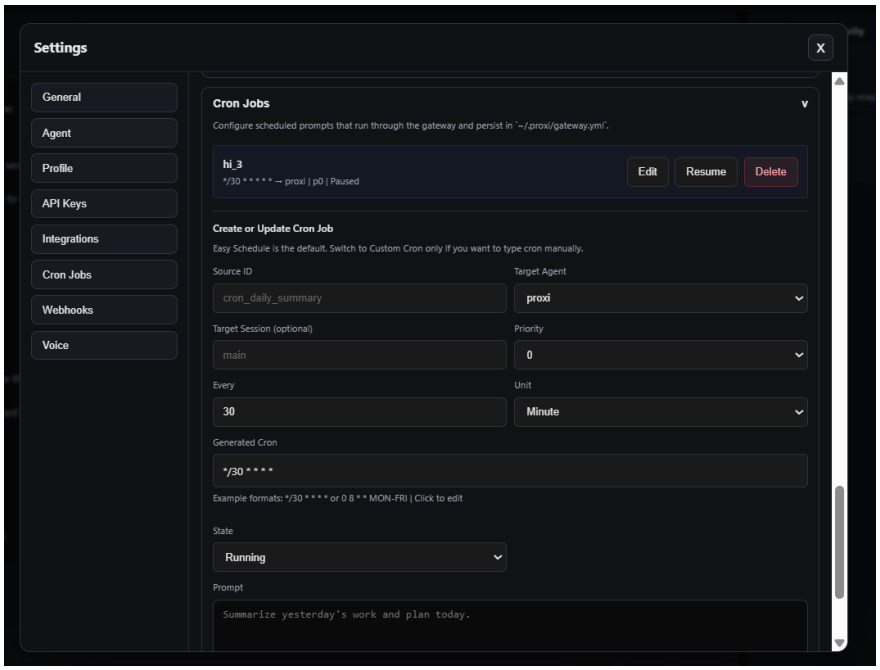

Figure 9: GUI Settings: Cron Jobs section.

5.7 Webhooks

Use webhooks for external event-driven automation.
For detailed relay and webhook configuration, see the README webhook section: ../../README.md#discord-integration-relay---gateway

5.7.1 Key Fields

- Source ID
- Prompt Template
- Target Agent and Target Session
- Priority and Paused state
- Optional secret reference

5.7.2 Suggested Rollout

1. Start with a simple template
2. Send one synthetic test event
3. Confirm output in chat/activity panel

4. Expand template complexity gradually

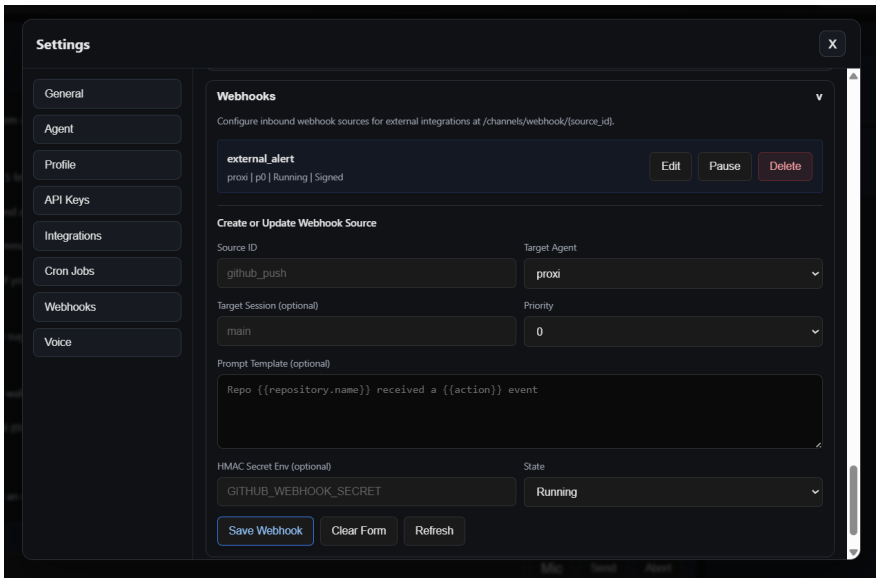

Figure 10: GUI Settings: Webhooks section.

5.8 LLM Dependency Note

Provider/model switching lives in **General**. Ensure the corresponding provider key exists in API Keys (for example `OPENAI_API_KEY`).

5.9 Voice

Use this section to configure wake-word listening, speech capture behavior, and browser text-to-speech playback.

5.9.1 What You Can Configure

- **Wake word**: enables or disables the browser-based listener used to detect "hey proxi"
- **Auto-stop sensitivity**: sets how many seconds of silence are required before Proxi stops recording
- **Auto-send after silence**: controls whether Proxi sends the transcript automatically when recording stops
- **Start-listening beep**: toggles the audible beep when voice capture begins

- **Text to speech**: enables or disables spoken assistant replies
- **Voice / accent**: selects the browser voice used for narration
- **Narration speed**: adjusts how fast spoken replies are read
- **Preview Voice**: plays a short sample with the current voice settings

5.9.2 Suggested Usage

- Turn on wake word if you want hands-free interaction in the GUI
- Leave auto-send on for the fastest voice-first workflow
- Use Preview Voice after picking a new browser voice or changing speed
- Keep TTS off if you only want transcription and do not want spoken replies

5.9.3 Main Screen Voice Controls

The GUI chat composer also shows the current voice state beside the session info, and the Mic button starts or stops the current recording session.

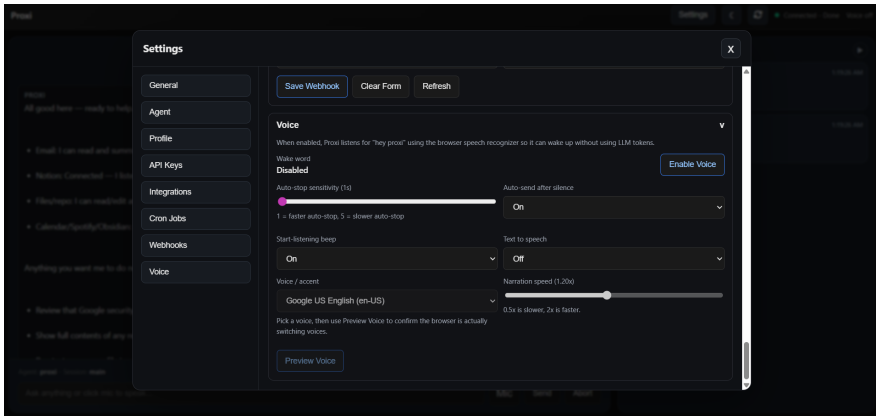

Figure 11: GUI Settings: Voice section.

5.10 MCP Setup by Integration

This section explains setup for each requested MCP.

5.10.1 Google Calendar MCP

Required Keys

```
GMAIL_CLIENT_ID
GMAIL_CLIENT_SECRET
```

Important Note
Google Calendar uses the same Google OAuth credentials as Gmail in this project.

Setup from Google Cloud Platform (GCP)

1. Open Google Cloud Console and create/select a project
2. Enable **Google Calendar API** and **Gmail API**
3. Configure OAuth consent screen (External/Internal as needed)
4. Create OAuth credentials as **Desktop App**
5. Copy the generated Client ID and Client Secret into:
   - `GMAIL_CLIENT_ID`
   - `GMAIL_CLIENT_SECRET`

First-Time Authorization Behavior
On first use of Google Calendar or Gmail tools, your browser opens Google sign-in and consent pages. After you approve access, Proxi stores a token file for future requests so you are not prompted every time.

Validation Prompt
"List my upcoming calendar events for tomorrow."

5.10.2 Gmail MCP

Required Keys

```
GMAIL_CLIENT_ID
GMAIL_CLIENT_SECRET
```

Setup Note
Gmail uses the same GCP OAuth setup described in **Google Calendar MCP** above. You only need one Google OAuth client for both integrations.

First-Time Authorization Behavior
On first Gmail action, browser-based Google consent is required once to generate/store token credentials.

Validation Prompt
"Show my latest unread emails."

5.10.3 Notion MCP

Required Keys

```
NOTION_API_KEY
NOTION_PARENT_PAGE_ID
```

Where to Get the Notion Values

1. Go to Notion integrations (My Integrations) and create an internal integration
2. Copy the integration secret into `NOTION_API_KEY`
3. Open the target parent page in Notion and copy its page ID from the URL
4. Save that ID as `NOTION_PARENT_PAGE_ID`

Also share the target page with your integration, otherwise API calls can fail with permissions errors.

Validation Prompt
"Create a new Notion note called Weekly Plan."

5.10.4 Obsidian MCP

Required Inputs
This MCP typically uses vault path/configuration rather than cloud API secrets.

Vault Path Setup
Set your Obsidian vault path in configuration/environment used by your Proxi runtime.

Example vault path values:

- Windows: `C:/Users/YOUR_USERNAME/Documents/ObsidianVault`
- macOS: `/Users/YOUR_USERNAME/Documents/ObsidianVault`
- Linux: `/home/YOUR_USERNAME/Documents/ObsidianVault`

Ensure the path exists and Proxi has permission to read/write notes there.

Validation Prompt
"Create an Obsidian note named Meeting Notes in my vault."

5.10.5 Spotify MCP

Required Keys

```
SPOTIFY_CLIENT_ID
SPOTIFY_CLIENT_SECRET
```

Setup from Spotify Developer Dashboard

1. Open Spotify Developer Dashboard and create an app
2. Copy app Client ID into `SPOTIFY_CLIENT_ID`
3. Copy app Client Secret into `SPOTIFY_CLIENT_SECRET`
4. Add the redirect URI used by your project (for example `http://127.0.0.1:8888/callback`)

First-Time Authorization Behavior
On first Spotify action, a browser window opens for Spotify login/consent. After approval, a token file is generated and reused for subsequent requests.

Validation Prompt
"Play my focus playlist on Spotify."

5.10.6 Weather MCP

Required Inputs
No API key is required in this setup. Weather uses the built-in Open-Meteo integration.

Setup Note
Enable **Weather** in **Settings > Integrations**. Accurate location and timezone settings in your profile help weather responses stay relevant.

Validation Prompt
"What is the weather in Hamilton, ON today?"

5.10.7 Web MCP

Required Inputs
No API key is required in this setup. Web uses Proxi's built-in web search integration.

Setup Note
Enable **Web** in **Settings > Integrations** when you want web search and browser-style lookup tools available to the assistant.

Validation Prompt
"Search the web for the latest news about OpenAI."

5.11 Example Setup: Complete Key and MCP Flow

5.11.1 Step 1: Add Keys

Add all required key names in **Settings > API Keys**:

```
OPENAI_API_KEY
GMAIL_CLIENT_ID
GMAIL_CLIENT_SECRET
NOTION_API_KEY
NOTION_PARENT_PAGE_ID
DISCORD_BOT_TOKEN
PROXI_DISCORD_COMMAND_PREFIX
SPOTIFY_CLIENT_ID
SPOTIFY_CLIENT_SECRET
```

Use `!proxi` as the value for `PROXI_DISCORD_COMMAND_PREFIX`.

5.11.2 Step 2: Enable Integrations

Enable these integrations in **Settings > Integrations**:

- Google Calendar
- Gmail
- Notion
- Obsidian
- Spotify
- Weather
- Web

5.11.3 Step 3: Validate One-by-One

Run one prompt per MCP to verify setup before moving to automation.

6 Discord Setup

This section contains the full Discord relay configuration and usage flow.

6.1 Configure Discord Source in Gateway

Add or update a source in your gateway configuration:

```
sources:
  discord:
type: discord
target_agent: work
target_session: discord
priority: 0
paused: false
# Optional:
# discord_command_prefix: /proxi
# discord_allow_plain: false
# discord_session_mode: fixed   # fixed | channel | user
# discord_agent_overrides: {}
```

Session mode behavior:

- `fixed`: all Discord messages for this source use one session
- `channel`: each Discord channel uses its own session
- `user`: each user in each channel uses its own session

6.2 Security Environment Variable (Recommended)

In the gateway process environment:

```
$env:DISCORD_WEBHOOK_SECRET = "your-shared-secret"
```

When set, `/channels/discord/webhook` requires `X-Signature-256` HMAC signatures.

6.3 Start Discord Relay

From project root (manual):

```
cd discord_relay
bun install
bun run start
```

Or via unified CLI:

```
uv run proxi discord
```

`proxi discord` requires a running gateway and exits if gateway is unreachable. Relay environment template: `discord_relay/.env.example`

6.4 Discord Command Usage

Default prefix is `/proxi`.

- `/proxi <task>`: enqueue a Proxi task
- `/proxi status`: show lane status for this channel/user session
- `/proxi abort`: abort active run in this session
- `/proxi switch <agent_id>`: route channel to a different agent
- `/proxi help`: show command help

To allow plain (non-prefixed) messages, set `discord_allow_plain: true` in gateway source config and `PROXI_DISCORD_ALLOW_PLAIN=1` in relay environment.

7 Troubleshooting

This section covers common issues and quick recovery steps.

7.1 Gateway or Interface Won't Connect

Symptoms

- GUI shows disconnected status
- TUI does not respond after startup
- `uv run proxi frontend` exits early

Fix

1. Start gateway first: `uv run proxi gateway start`
2. Start your interface after gateway is running
3. Ensure only one gateway process is active
4. If needed, stop and restart gateway, then relaunch TUI/GUI

7.2 Voice Input Is Misheard or Not Triggering

Symptoms

- Wake word does not activate listening
- Spoken text is inaccurate
- Mic button starts but no transcript appears

Fix

1. In GUI **Settings > Voice**, confirm wake word is enabled
2. Allow microphone permission in your browser
3. Reduce background noise and speak more clearly
4. Use the Mic button manually if wake-word activation is unreliable
5. Increase auto-stop sensitivity if recording ends too quickly

7.3 Integration Commands Fail (Gmail, Calendar, Spotify, etc.)

Symptoms

- Commands return auth errors
- Integration tools are unavailable
- Actions fail after keys were recently changed

Fix

1. Verify required API keys are present in **Settings > API Keys**
2. Confirm the integration is enabled in **Settings > Integrations**
3. Re-run first-time authorization flow when prompted in browser
4. Send one simple validation prompt for that integration
5. Restart gateway if tool availability appears stale

7.4 Cron or Webhook Jobs Do Not Run

Symptoms

- Scheduled prompts never trigger
- Webhook calls are accepted but do not produce actions

Fix

1. Confirm the source/job is not paused
2. Validate cron format and target agent/session values
3. Re-check webhook secret configuration if signatures are enabled
4. Inspect gateway logs for source-specific errors

7.5 Setup or Dependency Problems

Symptoms

- Missing dependencies after pulling new changes
- Commands fail due to missing runtime packages

Fix

1. From repository root run: `uv sync`
2. Re-run setup: `uv run proxi setup`
3. Retry gateway startup and then launch your selected interface

8 Quick End-to-End Flow

1. Start gateway once: `uv run proxi gateway start`
2. Launch TUI (`uv run proxi`) or GUI (`uv run proxi frontend`)
3. In GUI settings, configure General and Agent/Profile first
4. Add API keys and then enable required integrations
5. Add a cron job using Easy Schedule (Every + Unit), or use manual cron if needed
6. Send test prompts to confirm integrations and automation are working